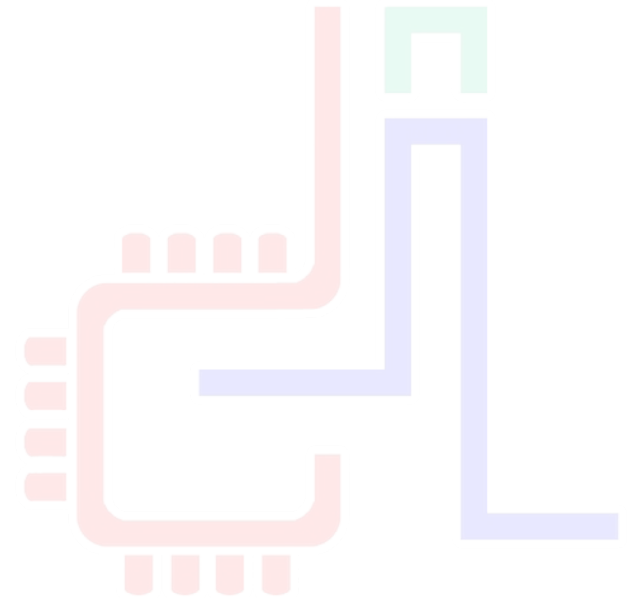


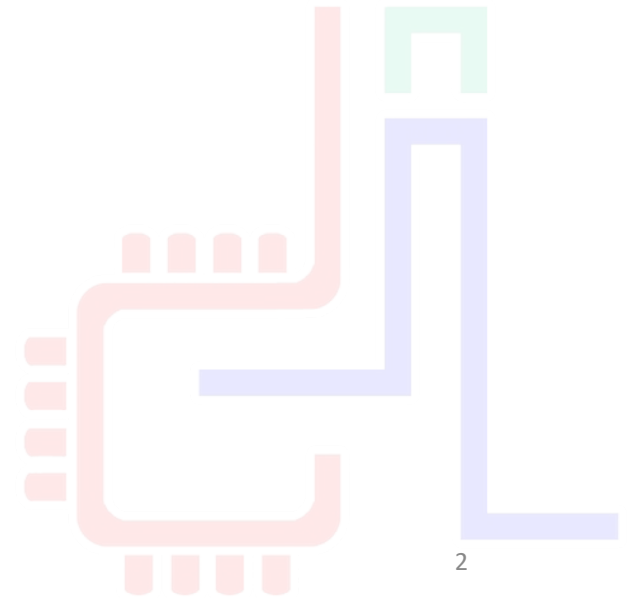
Framework BigDATA

TAREK BEN MENA





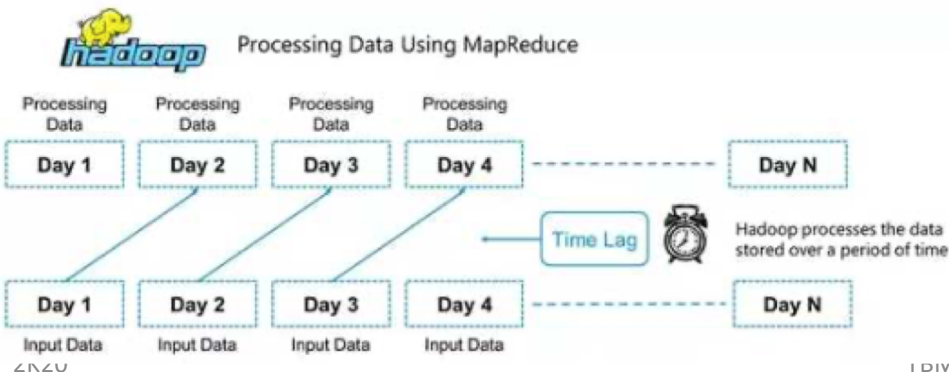
ANALYSER EN TEMPS RÉEL



Batch Processing / Stream Processing

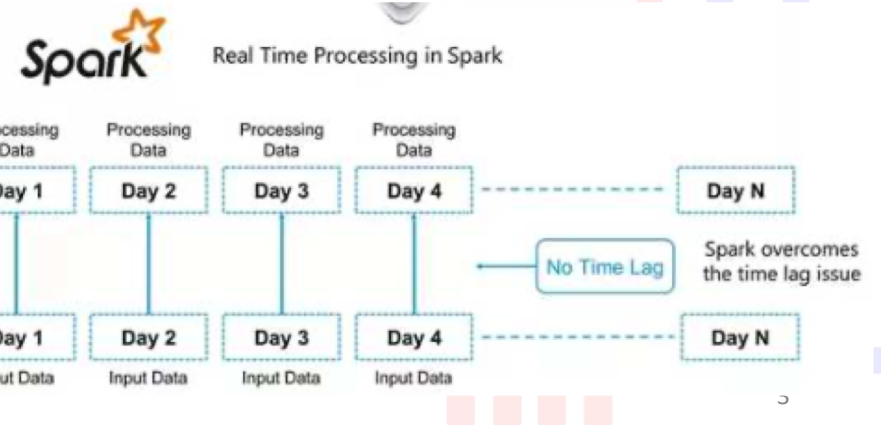
- **Batch Processing** (Traitement par lots):

- Traitement de blocs de données **déjà stockés** sur une période données.
- Par exemple, traiter toutes les transactions effectuées par une entreprise financière en une semaine.



- **Stream Processing** (Traitement de flux):

- les **données sont liées** avec un début et une fin dans un traitement qui se termine après le traitement de données finies
- traitement de flux de données sans fin arrivant **en temps réel** de façon continue pendant des jours, des mois, des années et à jamais.



Stream Processing

- 2 approches pour mettre en place un **Framework Streaming**:
 - **Native Streaming** (Real Time Processing)
 - Chaque enregistrement entrant est traité dès son arrivée, **sans attendre les autres**.
 - Exemples: Storm, Flink, Kafka Streams, Samza.
 - **Micro Batch Processing** (Micro Batching)
 - Les enregistrements entrants toutes les quelques secondes **sont mis en lots**, puis traités **en un seul mini-lot** avec un délai de quelques secondes.
 - Exemples: **Spark Streaming**, Storm-Trident.



Spark Streaming : Introduction

- **Spark** permet de traiter des données qui sont figées à un instant T.
- **Spark Streaming** permet de traiter des flux de données qui arrivent en continu,
- Avec Spark Streaming,
 - un contexte est initialisé avec une durée.
 - Le framework va accumuler des données pendant cette durée puis produire un petit RDD
- Micro-batches : cycle Accumulation / Production de RDD se reproduira jusqu'à à l'arrêt du programme.



Spark Stream vs Apache Storm

- **Storm** offre un traitement **en temps-réel** des évènements tandis que
 - **Spark Streaming** ajoutera un **délai** entre l'arrivée d'un message et son traitement.
 - **Spark Streaming** offre une garantie de traitement des messages
 - en **exactly once** (conditions normales) : chaque message est délivré une et une seule fois au programme, **sans perte** de messages,
 - et **at least once** (conditions dégradées) : message peut être délivré plusieurs fois, mais toujours **sans pertes**.
 - **Storm** permet de régler le niveau de garantie
 - mais, pour optimiser les performances, le mode **at most once** doit être utilisé : chaque message est délivré au maximum une fois **mais des pertes sont possibles**.
- L'API de Spark Streaming est **identique à l'API** classique de Spark.
 - manipulation des flux de données de la même manière que des données figées.

Source de données

- Spark Streaming traite des données qui arrivent en continu
- Les sources sont de 2 Catégories
 - **Sources basiques** (directe) via API : Système de fichier, une socket réseau
 - **Sources Avancées** via des classes extras : Kafka, Kenesis
- **Exemple**
 - une socket TCP (via `sc.socketStream` ou `sc.socketTextStream`)
 - des messages depuis [Kafka](#)
 - des logs depuis [Flume](#)
 - des fichiers depuis [HDFS](#) (pour monitorer la création de nouveaux fichiers uniquement)
 - une file MQ (type ZeroMQ)
 - des tweets depuis Twitter (utilise l'API [Twitter4J](#))
 - Il est également possible d'implémenter une source de données sur mesure en étendant la classe [Receiver](#).

1^{er} Exemple

```
from operator import add
from time import sleep
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

if __name__ == "__main__":
    sc = SparkContext(appName="Pyspark Stream")
    ssc = StreamingContext(sc, 1)
    #Input Data
    rddQueue = []
    for i in range(5):
        rddQueue += [ssc.sparkContext.parallelize([i, i+1])]

    inputStream = ssc.queueStream(rddQueue)

    inputStream.map(lambda x: "Input: " + str(x)).pprint()
    inputStream.reduce(add).map(lambda x: "Output: " + str(x)).pprint()

    ssc.start()
    sleep(5)
    ssc.stop(stopSparkContext=True, stopGraceFully=True)
```

1^{er} Exemple

```
from operator import add
from time import sleep
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

if __name__ == "__main__":
    sc = SparkContext(appName="Pyspark Stream")
    ssc = StreamingContext(sc, 1) ←
    #Input Data
    rddQueue = []
    for i in range(5):
        rddQueue += [ssc.sparkContext.parallelize([i, i+1])] ←

    inputStream = ssc.queueStream(rddQueue) ←

    inputStream.map(lambda x: "Input: " + str(x)).pprint()
    inputStream.reduce(add).map(lambda x: "Output: " + str(x)).pprint()

    ssc.start()
    sleep(5) ←
    ssc.stop(stopSparkContext=True, stopGraceFully=True)
```

Initialisation d'un contexte avec un temps de latence d'1 sec

[[0, 1], [1, 2], [2, 3], [3, 4], [4, 5]]

Créer un flux d'entrée de type (**DStream**) à partir d'une file ou liste de RDD

Suspend l'exécution du thread courant pour une durée de 5 sec

```
ssc.start()           # Start the computation
ssc.awaitTermination(5) # Wait 5 second for the
                        # computation to terminate
```

Autre syntaxe équivalente

1^{er} Exemple

```
from operator import add
from time import sleep
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

if __name__ == "__main__":
    sc = SparkContext(appName="Pyspark Stream")
    ssc = StreamingContext(sc, 1)
    #Input Data
    rddQueue = []
    for i in range(5):
        rddQueue += [ssc.sparkContext.parallelize([i, i+1])]

    inputStream = ssc.queueStream(rddQueue)

    inputStream.map(lambda x: "Input: " + str(x)).pprint()
    inputStream.reduce(add).map(lambda x: "Output: " + str(x)).pprint()

    ssc.start()
    sleep(5)
    ssc.stop(stopSparkContext=True, stopGraceFully=True)
```

```
ssc.start()           # Start the computation
ssc.awaitTermination() # Wait for the computation to
terminate
```

Time: 2020-11-08 14:31:08

Input: 0
Input: 1

Time: 2020-11-08 14:31:08

Output: 1

Time: 2020-11-08 14:31:09

Input: 1
Input: 2

Time: 2020-11-08 14:31:09

Output: 3

Time: 2020-11-08 14:31:10

Input: 2
Input: 3

Time: 2020-11-08 14:31:10

Output: 5

Time: 2020-11-08 14:31:11

Input: 3
Input: 4

Time: 2020-11-08 14:31:11

Output: 7

Time: 2020-11-08 14:31:12

Input: 4
Input: 5

Time: 2020-11-08 14:31:12

Output: 9

2ème Exemple

```
from operator import add
from time import sleep
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

if __name__ == "__main__":
    sc = SparkContext(appName="Pyspark Stream")
    ssc = StreamingContext(sc, 1)
    #Input Data
    inputData = [
        [1, 2, 3],
        [0],
        [4, 4, 4],
        [0, 0, 0, 25],
        [1, -1, 10]
    ]
    rddQueue = []
    for datum in inputData:
        rddQueue += [ssc.sparkContext.parallelize(datum)]

    inputStream = ssc.queueStream(rddQueue)
    inputStream.reduce(add).pprint()

    ssc.start()
    sleep(5)
    ssc.stop(stopSparkContext=True, stopGraceFully=True)
```

Time: 2020-11-08 15:06:36

6

Time: 2020-11-08 15:06:37

0

Time: 2020-11-08 15:06:38

12

Time: 2020-11-08 15:06:39

25

Time: 2020-11-08 15:06:40

10

Time: 2020-11-08 15:06:41

StreamingContext

- Pour initialiser programme Spark Streaming il faut créer un objet `StreamingContext` en précisant un temps de latence en seconde

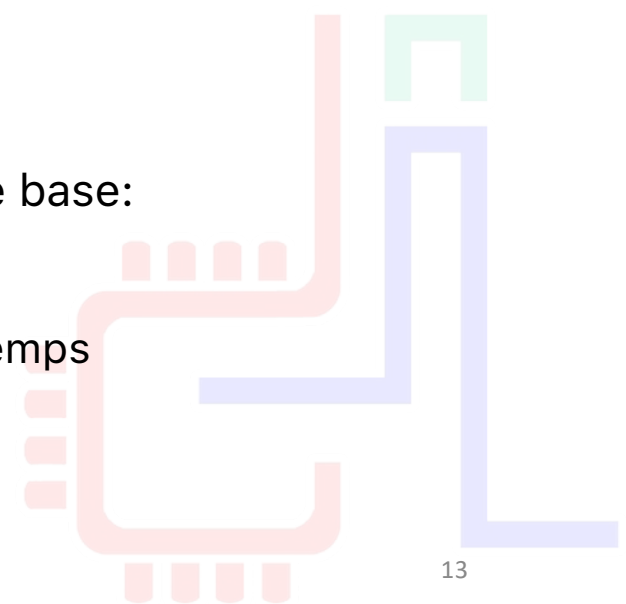
```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

sc = SparkContext(appName="Pyspark Stream")
ssc = StreamingContext(sc, 1)
```

- Après
 1. Définir une source d'entrée en créant un input `DStream` (Discretized Stream)
 2. Définir le traitement du stream en appliquant des transformations et une des opérations de sortie de `DStream`
 3. Lancer la réception des données et le traitement avec `streamingContext.start()`.
 4. Attendre que le processus s'arrête avec `streamingContext.awaitTermination()`.
 5. Le processus peut être manuellement arrêter avec `streamingContext.stop()`.

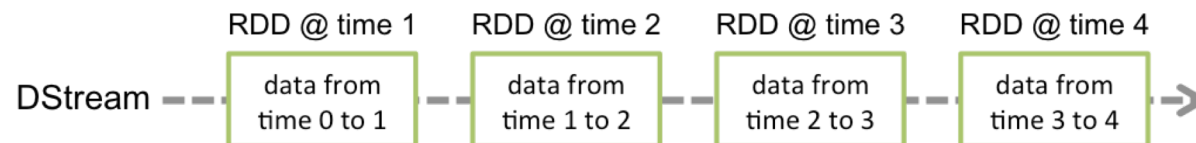
DStream : Discretized Stream

- Classe d'abstraction de base de Spark Streaming, représente est une séquence continue de **RDD** (du même type) représentant un **flux continu de données**.
- Les DStreams peuvent être
 - créés à partir de données en direct (telles que des données provenant de sockets TCP, etc.) à l'aide d'un **StreamingContext**
 - générés en transformant des **DStreams** existants à l'aide d'opérations telles que **map**, **window** et **reductionByKeyAndWindow**.
- Lorsqu'un programme Spark Streaming est en cours d'exécution,
 - Chaque **DStream** génère périodiquement un RDD,
 - soit à partir de données en direct,
 - soit en transformant le RDD généré par un DStream parent.
- DStreams se caractérise en interne par quelques propriétés de base:
 - Une **liste des autres DStreams** dont dépend le DStream
 - Un **intervalle de temps** auquel le DStream génère un RDD
 - Une **fonction qui génère un RDD** après chaque intervalle de temps
- [Documentation officielle](#)

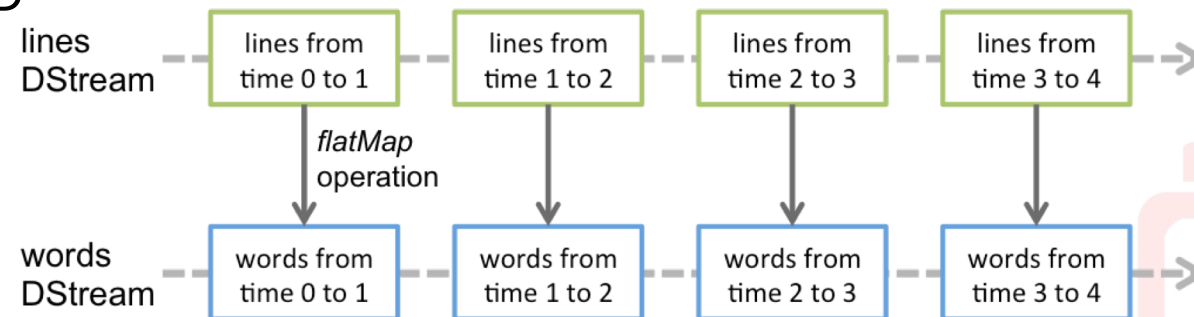


Discretized Stream : DStream

- Chaque RDD dans un DStream contient les données d'un certain intervalle



- Toute opération appliquée sur un DStream est tradatée et appliquée sur chaque RDD



DStream : Transformations

Transformation	Meaning
map (<i>func</i>)	Return a new DStream by passing each element of the source DStream through a function <i>func</i> .
flatMap (<i>func</i>)	Similar to map, but each input item can be mapped to 0 or more output items.
filter (<i>func</i>)	Return a new DStream by selecting only the records of the source DStream on which <i>func</i> returns true.
repartition (<i>numPartitions</i>)	Changes the level of parallelism in this DStream by creating more or fewer partitions.
union (<i>otherStream</i>)	Return a new DStream that contains the union of the elements in the source DStream and <i>otherDStream</i> .
count ()	Return a new DStream of single-element RDDs by counting the number of elements in each RDD of the source DStream.
reduce (<i>func</i>)	Return a new DStream of single-element RDDs by aggregating the elements in each RDD of the source DStream using a function <i>func</i> (which takes two arguments and returns one). The function should be associative and commutative so that it can be computed in parallel.
countByValue ()	When called on a DStream of elements of type K, return a new DStream of (K, Long) pairs where the value of each key is its frequency in each RDD of the source DStream.
reduceByKey (<i>func</i> , [<i>numTasks</i>])	When called on a DStream of (K, V) pairs, return a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function. Note: By default, this uses Spark's default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property <code>spark.default.parallelism</code>) to do the grouping. You can pass an optional <i>numTasks</i> argument to set a different number of tasks.
join (<i>otherStream</i> , [<i>numTasks</i>])	When called on two DStreams of (K, V) and (K, W) pairs, return a new DStream of (K, (V, W)) pairs with all pairs of elements for each key.
cogroup (<i>otherStream</i> , [<i>numTasks</i>])	When called on a DStream of (K, V) and (K, W) pairs, return a new DStream of (K, Seq[V], Seq[W]) tuples.
transform (<i>func</i>)	Return a new DStream by applying a RDD-to-RDD function to every RDD of the source DStream. This can be used to do arbitrary RDD operations on the DStream.
updateStateByKey (<i>func</i>)	Return a new "state" DStream where the state for each key is updated by applying the given function on the previous state of the key and the new values for the key. This can be used to maintain arbitrary state data for each key.

DStream : updateStateByKey

- L'opération `updateStateByKey` permet de conserver un **état** tout en le mettant continuellement à jour avec de nouvelles informations. Il effectue deux étapes.
 - Définir l'état - L'état peut être un type de données.
 - Définir la fonction de mise à jour de l'état - Spécifiez avec une fonction comment mettre à jour l'état en utilisant l'**état précédent** et les **nouvelles valeurs** d'un flux d'entrée.

```
def updateFunc(new_values, last_sum):  
    return sum(new_values) + (last_sum or 0)  
  
lines = ssc.socketTextStream(sys.argv[1], int(sys.argv[2]))  
running_counts = lines.flatMap(lambda line: line.split(" "))\  
                        .map(lambda word: (word, 1))\  
                        .updateStateByKey(updateFunc, initialRDD=initialStateRDD)  
  
running_counts.pprint()  
ssc.start()  
ssc.awaitTermination()
```

DStream : transformation

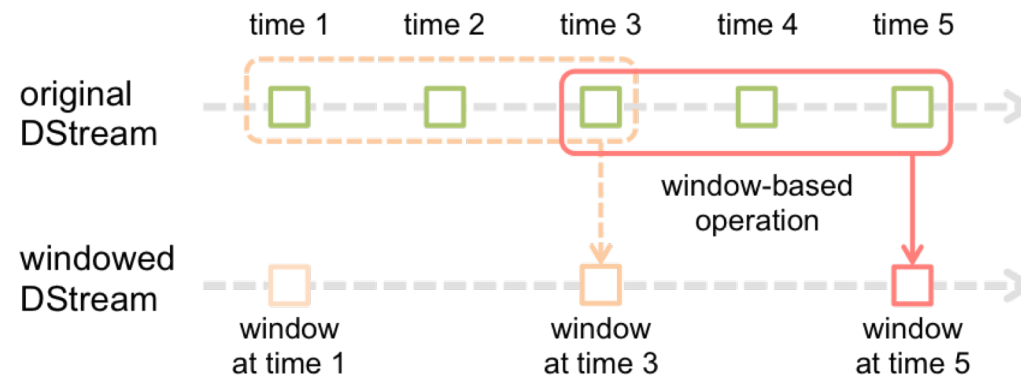
- L'opération de `transformation` permet d'appliquer des fonctions RDD-à-RDD sur un DStream.
 - utilisé pour appliquer toute opération RDD qui n'est dans l'API DStream.
 - Par exemple, `join` n'est pas dans l'API DStream.

```
spamInfoRDD = sc.pickleFile(...) # RDD containing spam information

# join data stream with spam information to do data cleaning
cleanedDStream = wordCounts.transform(lambda rdd: rdd.join(spamInfoRDD).filter(...))
```

DStream : opérations window

- Appliquer des transformations sur une fenêtre glissante de données.
- Chaque fois que la fenêtre glisse sur un DStream source, les RDD source (dans la fenêtre) **sont combinés** pour produire les RDD du DStream fenêtré.



- Toute opération de fenêtre doit spécifier **deux paramètres multiple du batch interval**.
 - *window length* - La durée de la fenêtre.
 - Figure : est appliquée sur les 3 dernières unités de temps de données
 - *Sliding interval* - L'intervalle auquel l'opération de fenêtre est effectuée.
 - Figure: glisse de 2 unités de temps

Dstream – Opérations window - exemple

- Appliquée sur les 3 dernières unités de temps de données
- Glisse deux unité de temps

```
from operator import add
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

if __name__ == "__main__":
    sc = SparkContext(appName="Pyspark Stream")
    ssc = StreamingContext(sc, 1)
    #Input Data
    rddQueue = []
    for i in range(100):
        rddQueue += [ssc.sparkContext.parallelize([i, i+1])]
    inputStream = ssc.queueStream(rddQueue).window(3,2)

    inputStream.map(lambda x: "Input: " + str(x)).pprint()
    inputStream.reduce(add).map(lambda x: "Output: " + str(x)).pprint()

    ssc.start()
    ssc.awaitTermination()
```

Time: 2021-01-04 16:45:59

Input: 0
Input: 1
Input: 1
Input: 2

Time: 2021-01-04 16:45:59

Output: 4

Time: 2021-01-04 16:46:01

Input: 1
Input: 2
Input: 2
Input: 3
Input: 3
Input: 4

Time: 2021-01-04 16:46:01

Output: 15

Time: 2021-01-04 16:46:03

Input: 3
Input: 4
Input: 4
Input: 5
Input: 5
Input: 6

Time: 2021-01-04 16:46:03

Output: 27

DStream : opérations window

■ Exemple

- générer des comptes de mots sur les 30 dernières secondes de données, toutes les 10 secondes.
- il faut appliquer l'opération `reduceByKey` sur les *paires* DStream de (mot, 1) sur les 30 dernières secondes de données avec la méthode `reduceByKeyAndWindow`.

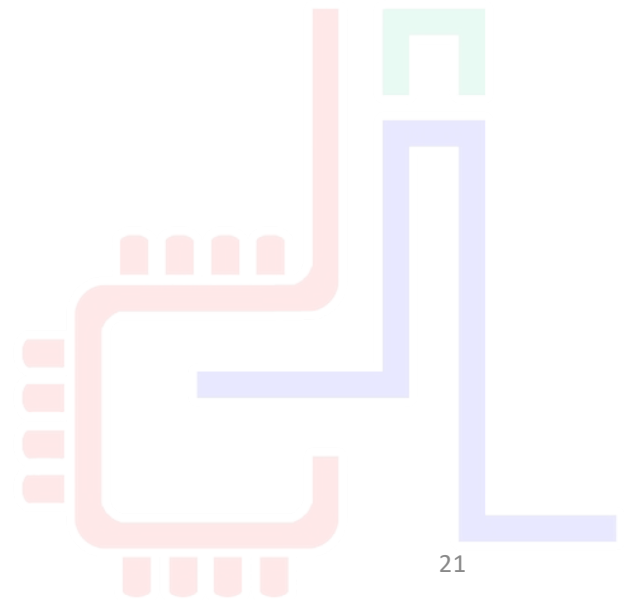
```
# Reduce last 30 seconds of data, every 10 seconds
windowedWordCounts = pairs.reduceByKeyAndWindow(lambda x, y: x + y, 30, 10)
```

Transformation	Meaning
<code>window(windowLength, slideInterval)</code>	Return a new DStream which is computed based on windowed batches of the source DStream.
<code>countByWindow(windowLength, slideInterval)</code>	Return a sliding window count of elements in the stream.
<code>reduceByWindow(func, windowLength, slideInterval)</code>	Return a new single-element stream, created by aggregating elements in the stream over a sliding interval using <i>func</i> . The function should be associative and commutative so that it can be computed correctly in parallel.
<code>reduceByKeyAndWindow(func, windowLength, slideInterval, [numTasks])</code>	When called on a DStream of (K, V) pairs, returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> over batches in a sliding window. Note: By default, this uses Spark's default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property <code>spark.default.parallelism</code>) to do the grouping. You can pass an optional <code>numTasks</code> argument to set a different number of tasks.

STRUCTURED STREAM

2K20

TBM

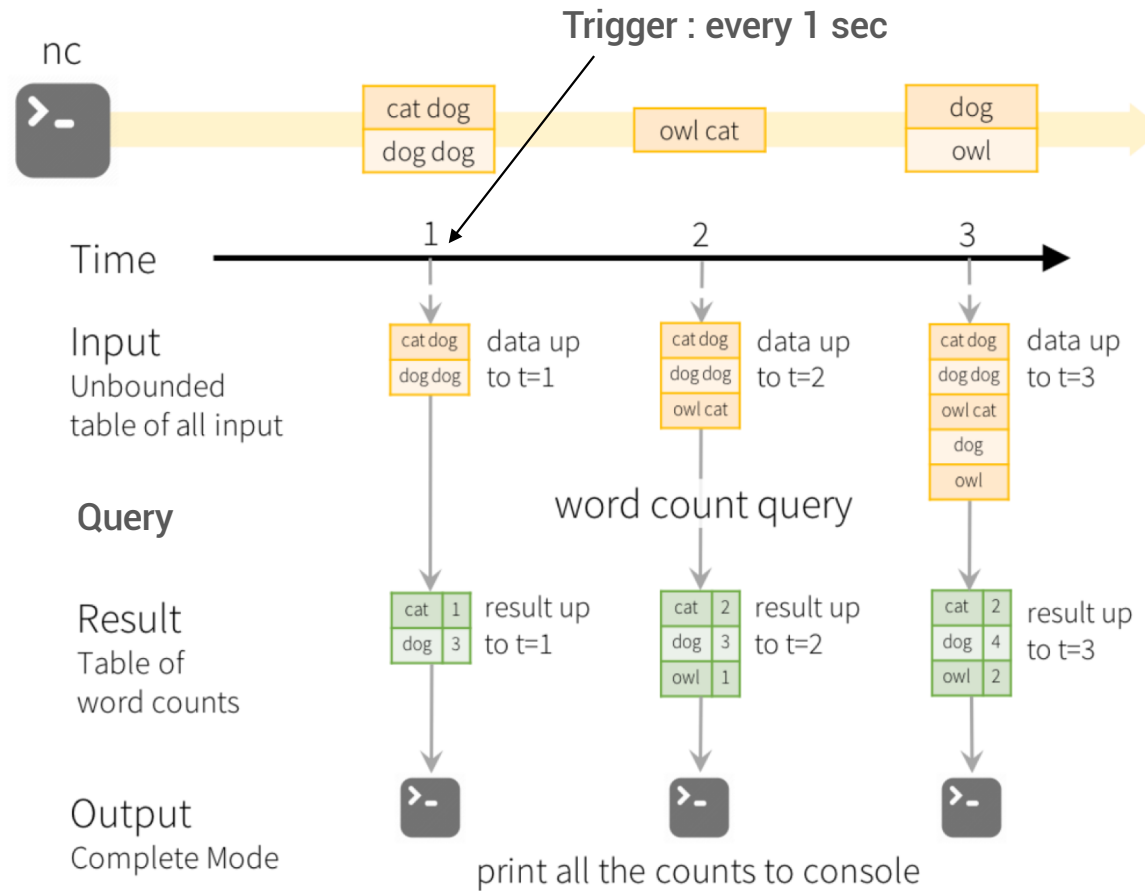


21

Structured Streaming

- API de streaming **de haut niveau** basée sur le moteur Spark SQL
 - Exécute le même calcul que les requêtes par lots dans les Datasets / DataFrames
 - Eventtime, Windowing, sessions, sources et récepteurs
- Unifie les requêtes en streaming, requêtes interactives et par lots
 - Agrège les données dans un flux, puis les diffuse à l'aide de JDBC
 - Ajouter, supprimer, modifier des requêtes au moment de l'exécution
 - Créer et appliquer des modèles ML
- Avantages par rapport à DStreams
 1. Traitement avec le **temps-événement**, **traitement des données tardives**
 2. Exactement la **même API** pour le **traitement par lots**, le streaming et l'interactivité
 3. Performances grâce aux optimisations SQL
 - Optimisations de plans logiques, Tungstène, Codegen, etc.
 - Gestion d'état plus rapide pour le traitement de flux avec état

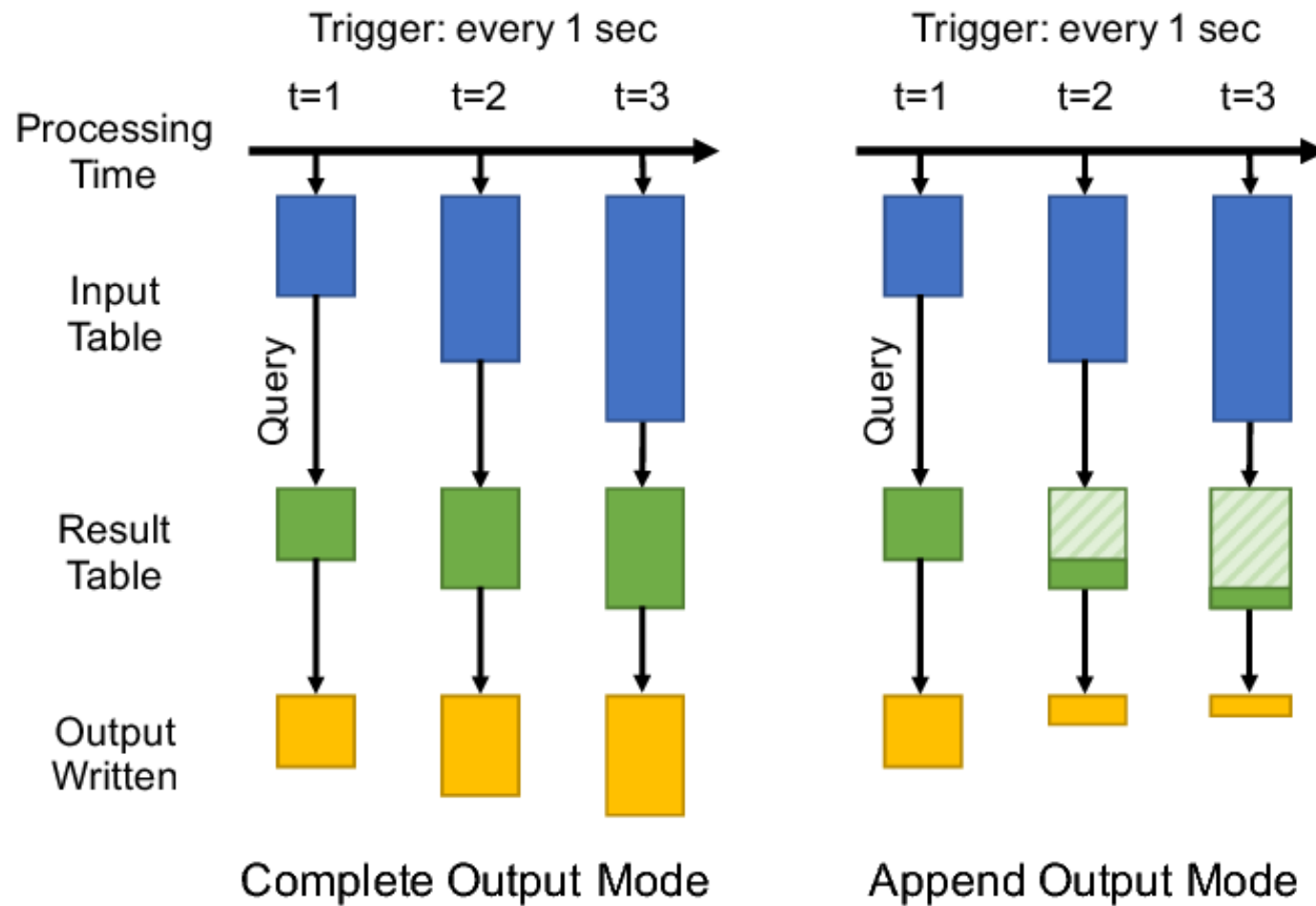
Programming Model



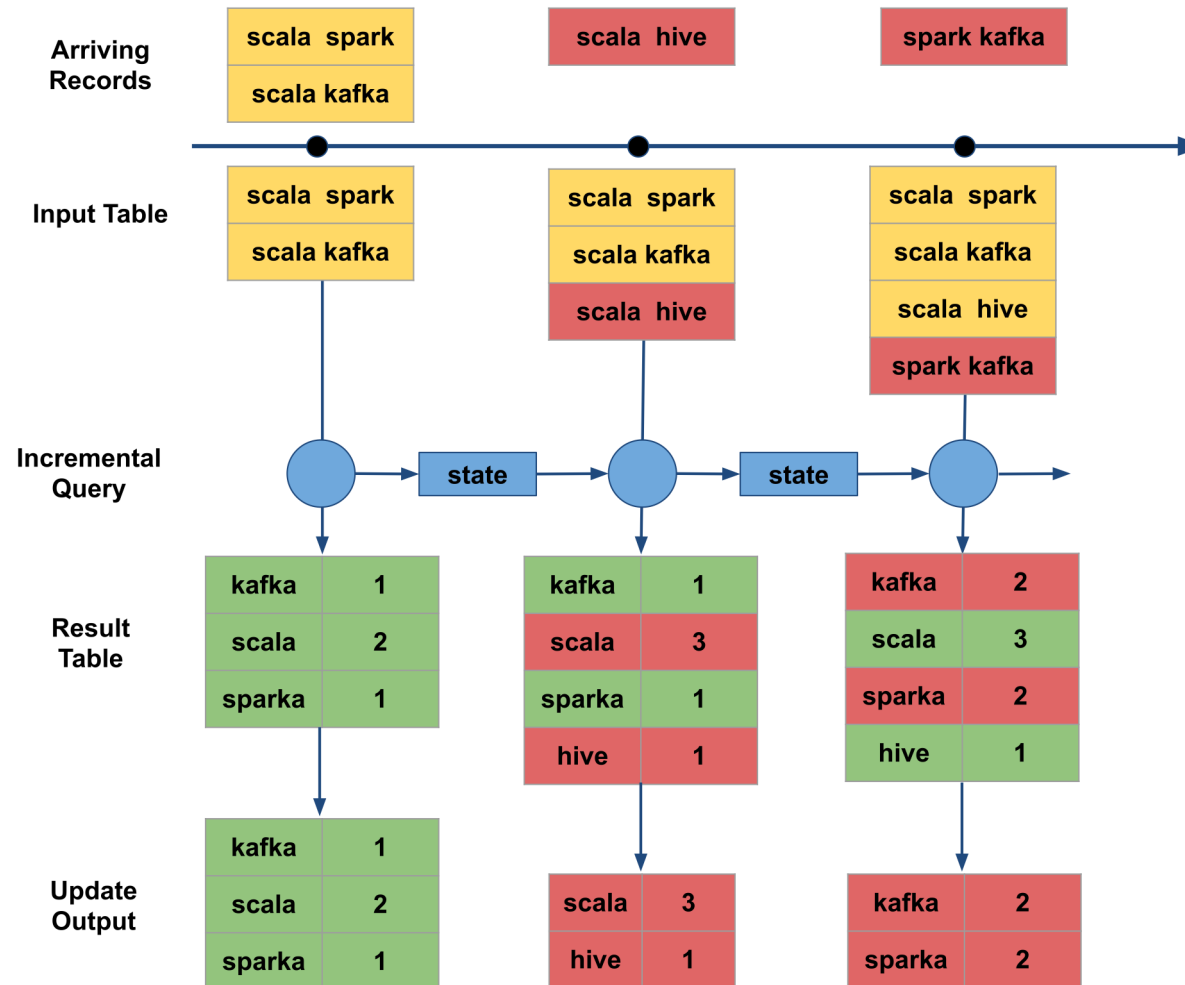
Modes de Sortie

- Il existe quelques types de modes de sortie.
- **Mode append** (par défaut) - seules les nouvelles lignes ajoutées à la table de résultats depuis le dernier **trigger** seront envoyées au récepteur.
 - Utilisé pour les requêtes où les lignes ajoutées à la table de résultats ne changeront jamais : `select`, `where`, `map`, `flatMap`, `filter`, `join`
 - garantit que chaque ligne ne sera sortie qu'une seule
- **Mode complet** - L'ensemble du tableau de résultats sera envoyé au récepteur après chaque **trigger**. Ceci est pris en charge pour les requêtes d'agrégation.
- **Mode update** - (Spark 2.1.1) Seules les lignes de la table de résultats qui ont été mises à jour depuis le dernier **trigger** seront envoyées au récepteur.

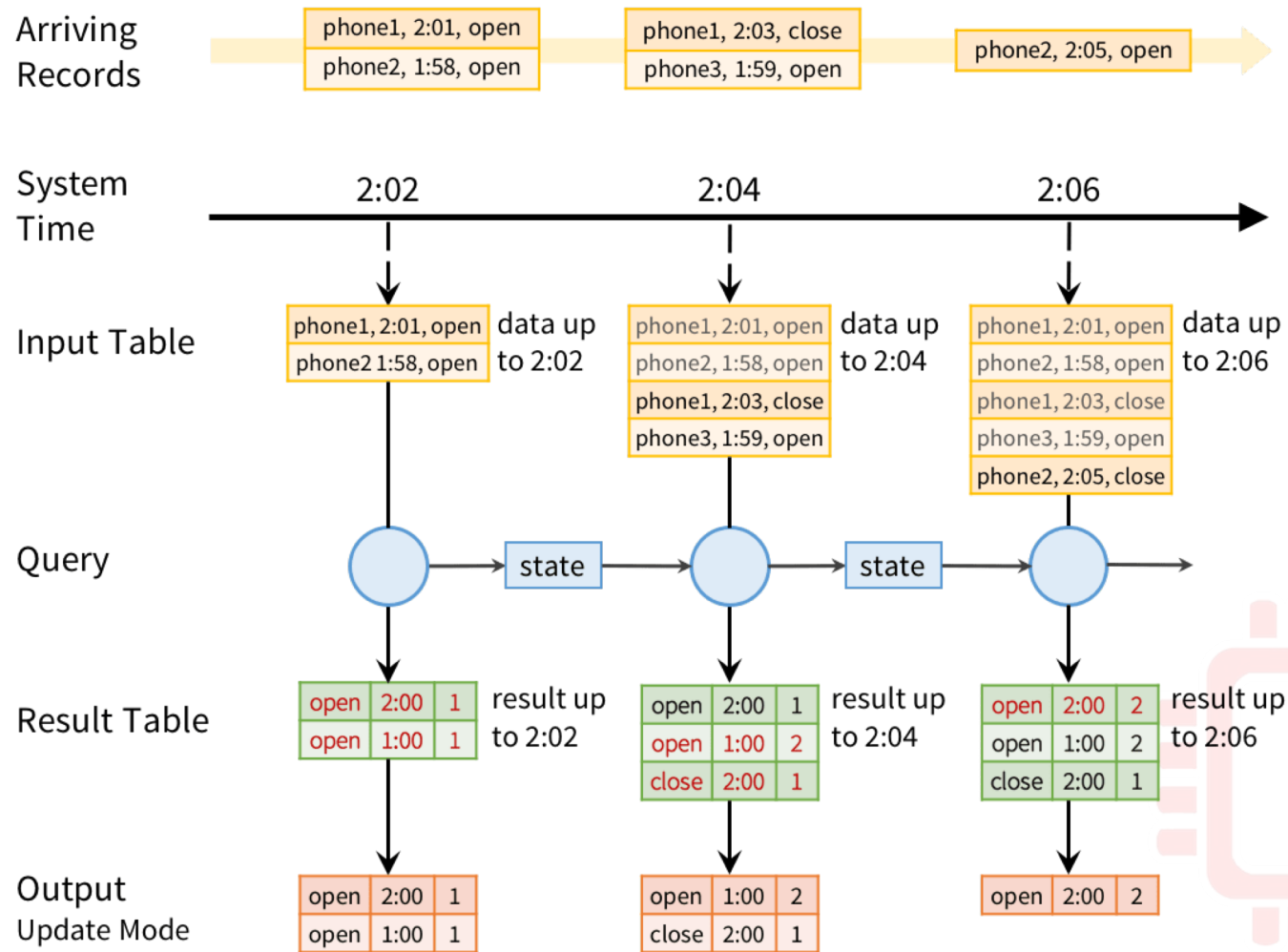
Modes de Sortie – Complete vs Append mode



Modes de Sortie – Update mode

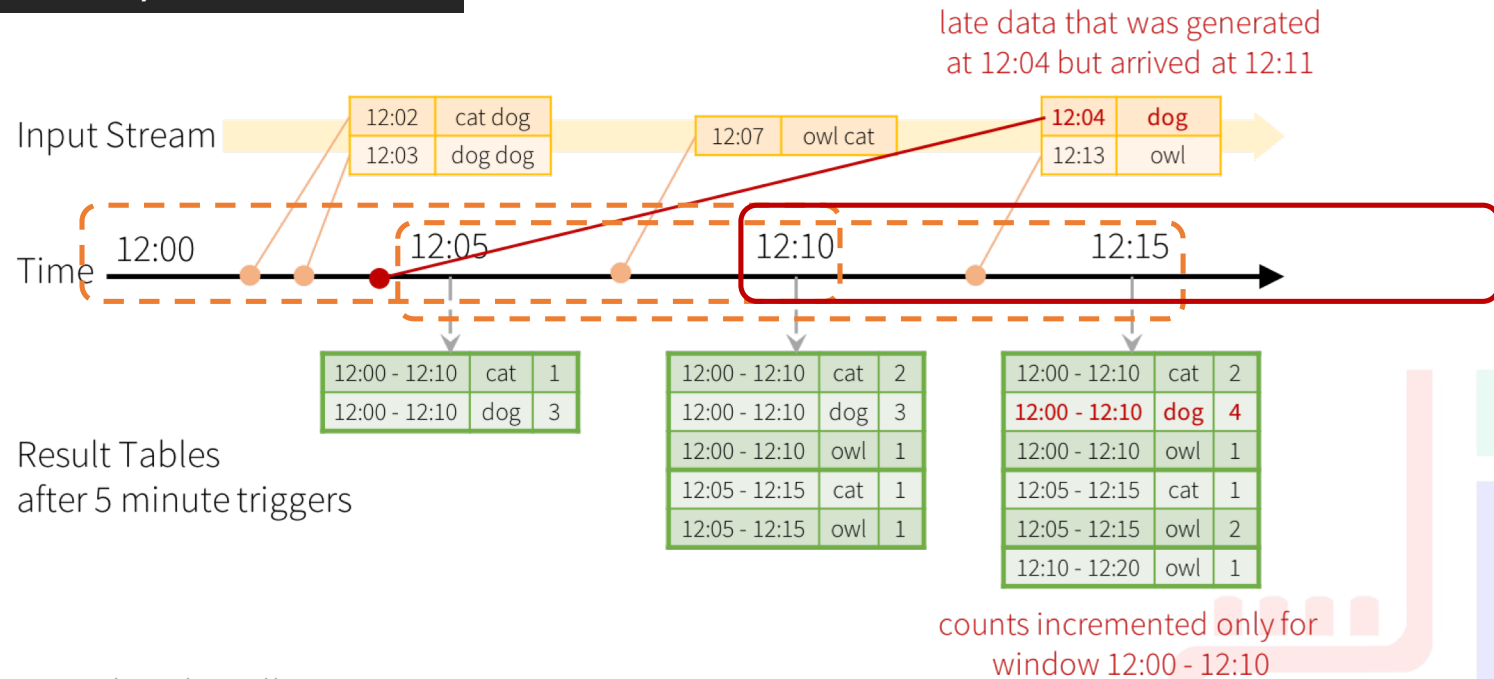


Modes de Sortie – Update mode – exemple 2



SStream - Gestion des données tardives

```
window("10 minutes", "5 minutes")
```



Late data handling in
Windowed Grouped Aggregation

Watermarking

- Pendant des jours, le système doit limiter la quantité d'états intermédiaires en mémoire.
 - un ancien agrégat peut être supprimé de l'état en mémoire (si l'application ne recevra plus de données en retard pour cet agrégat (en cas de watermark)
- Le **watermark** (Spark 2.1), permet au moteur de suivre automatiquement l'**event-time** courant dans les données et d'essayer de **nettoyer l'ancien état** en conséquence.
 - définir le watermark d'une requête en spécifiant la colonne d'heure de l'événement et le seuil de retard attendu avec **withWatermark()**.
- Pour une fenêtre se terminant au temps **T**, le moteur maintiendra l'état et permettra aux données tardives de mettre à jour l'état jusqu'au (**max event time seen by the engine - late threshold > T**).

Watermark

- En d'autres termes, les données **en retard dans le seuil** seront agrégées, mais les **postérieures au seuil** commenceront à être abandonnées

```
words = ... # streaming DataFrame of schema { timestamp: Timestamp, word: String }

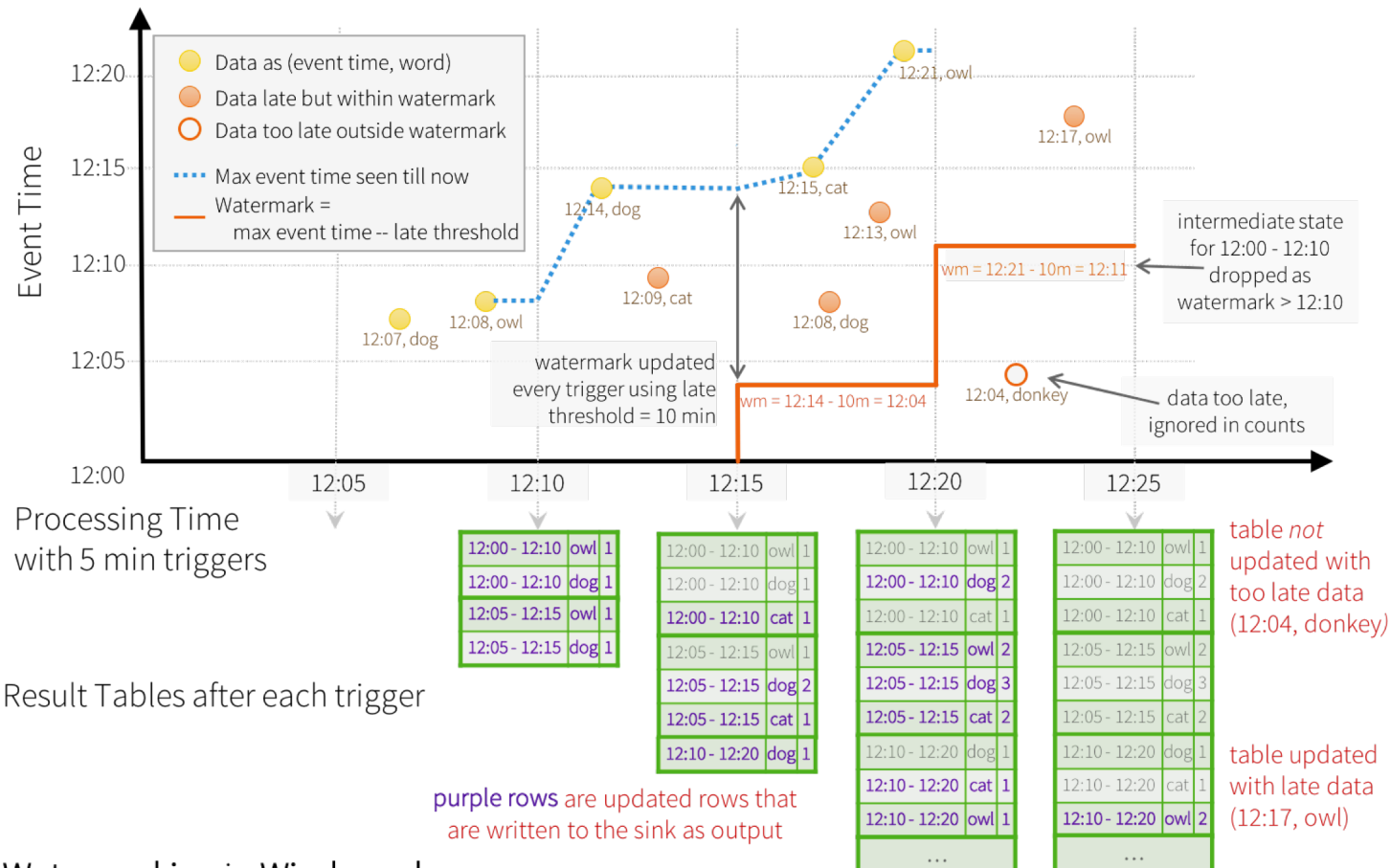
# Group the data by window and word and compute the count of each group
windowedCounts = words \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(
        window(words.timestamp, "10 minutes", "5 minutes"),
        words.word) \
    .count()
```

- **Exemple**

- Watermark de la requête défini sur la valeur de la **colonne** «**timestamp**», «**10 minutes**» comme le seuil de retard autorisé pour les données.
- Si cette requête est exécutée en mode de sortie de **Update**, le moteur continuera à mettre à jour le nombre d'une fenêtre dans le tableau des résultats jusqu'à ce que la fenêtre soit plus ancienne que le watermark, qui est en retard par rapport au time-event actuel dans la colonne «**timestamp**» de 10 minutes.

Watermarking

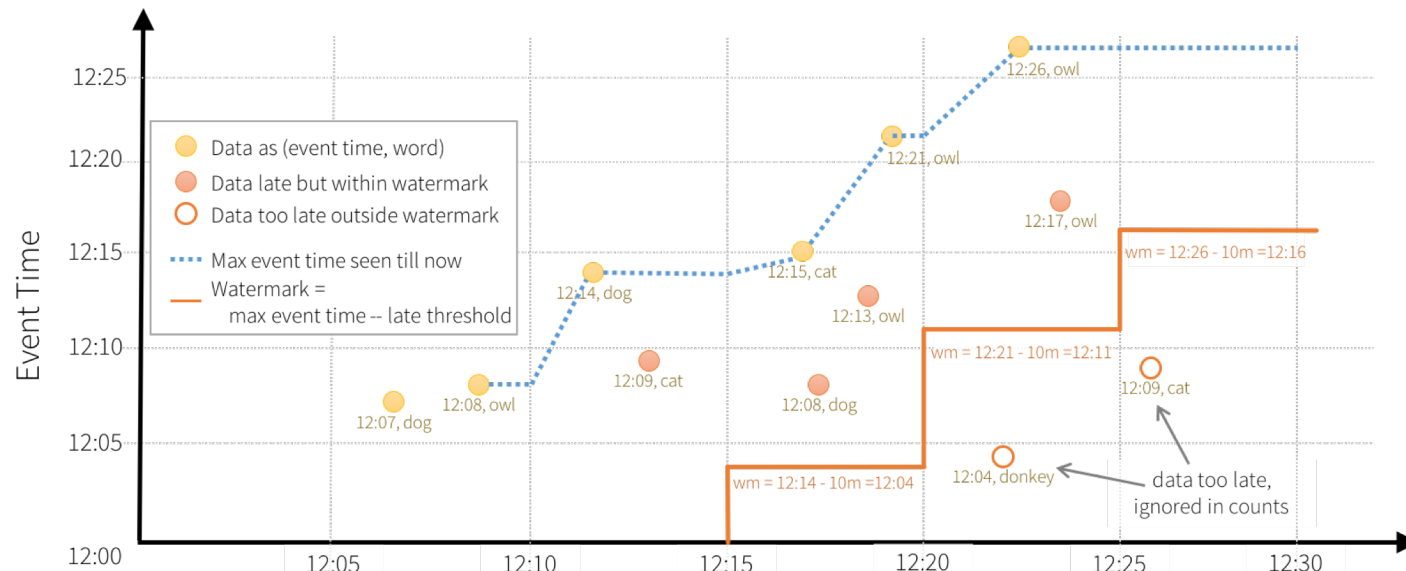
```
.withWatermark("timestamp", "10 minutes") \
.groupBy(
    window(words.timestamp, "10 minutes", "5 minutes"),
    words.word) \
```



Watermarking in Windowed
Grouped Aggregation with Update Mode

TBM

Watermarking



Processing Time
with 5 min triggers

partial counts for window 12:00 - 12:10 maintained as internal state while waiting for late data, so not yet added to result table

final counts for 12:00 - 12:10 added to table when watermark > 12:10, late data counted, and intermediate state for window dropped

12:00 - 12:10	owl	1
12:00 - 12:10	cat	1
12:00 - 12:10	dog	2

12:00 - 12:10	owl	1
12:00 - 12:10	cat	1
12:00 - 12:10	dog	2
12:05 - 12:15	owl	2
12:05 - 12:15	cat	2
12:05 - 12:15	dog	3

Result Tables after each trigger

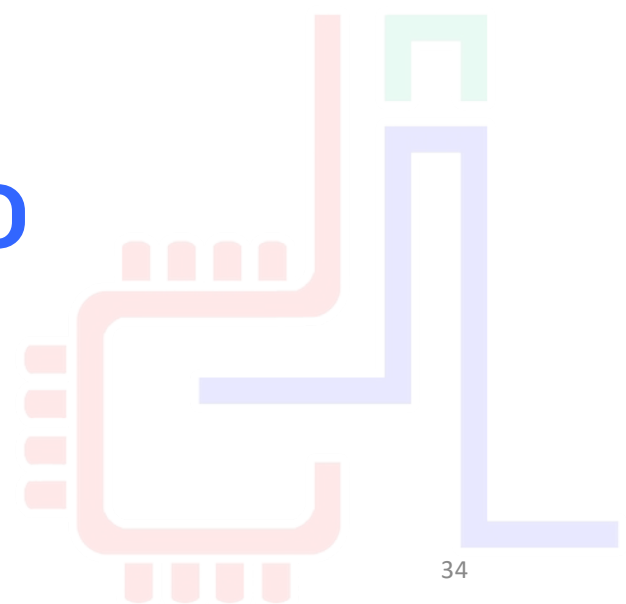
Watermarking in Windowed
Grouped Aggregation with Append Mode

Conclusion avec d'autres moteurs

Property	Structured Streaming	Spark Streaming	Apache Storm	Apache Flink	Kafka Streams	Google Dataflow
Streaming API	incrementalize batch queries	integrates with batch	separate from batch	separate from batch	separate from batch	integrates with batch
Prefix Integrity Guarantee	✓	✓	✗	✗	✗	✗
Internal Processing	exactly once	exactly once	at least once	exactly once	at least once	exactly once
Transactional Sources/Sinks	✓	some	some	some	✗	✗
Interactive Queries	✓	✓	✗	✗	✗	✗
Joins with Static Data	✓	✓	✗	✗	✗	✗

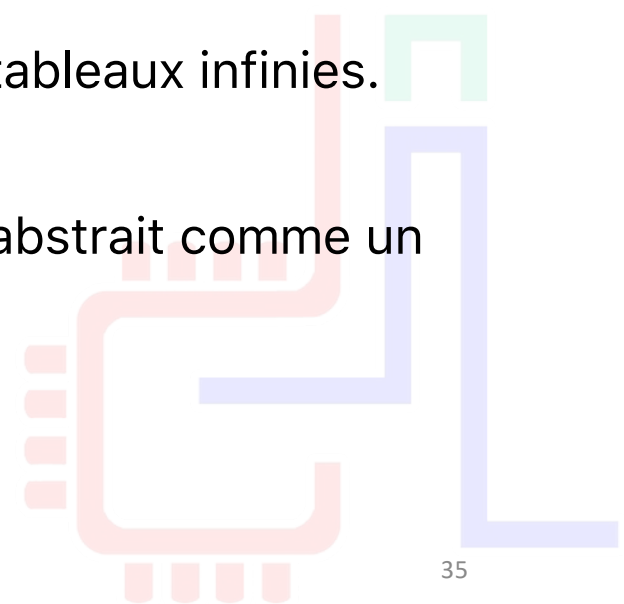


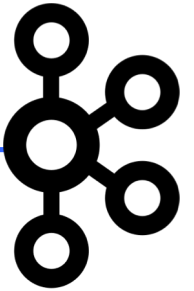
TUTOTRIAL : SPARK STRUCTURED STREAMING WITH KAFKA



Spark Structured Streaming

- Spark Structured Streaming est un nouveau moteur de traitement stream introduit avec Apache Spark 2.
- Il est construit sur le moteur Spark SQL (même API) et utilise le modèle Spark DataFrame.
- Les données de streaming sont modélisées comme des tableaux infinies.
- L'API permet l'exécution de requêtes SQL sur un stream abstrait comme un tableau.





Apache Kafka

- Développé par LinkedIn puis par Apache Software Foundation (2014)
- C'est un système unifié (Agent de Message), en **temps réel** à latence faible pour la manipulation de flux de données
 - mise en place de « data pipeline » temps réel
 - système d'échange dans les architectures micro services
 - supervision temps réel et IoT
- Ingère et diffuse des données massive, couplé à un framework de Data Stream Processing (Spark Stream) il permet le traitement complexe et en temps réel des données.

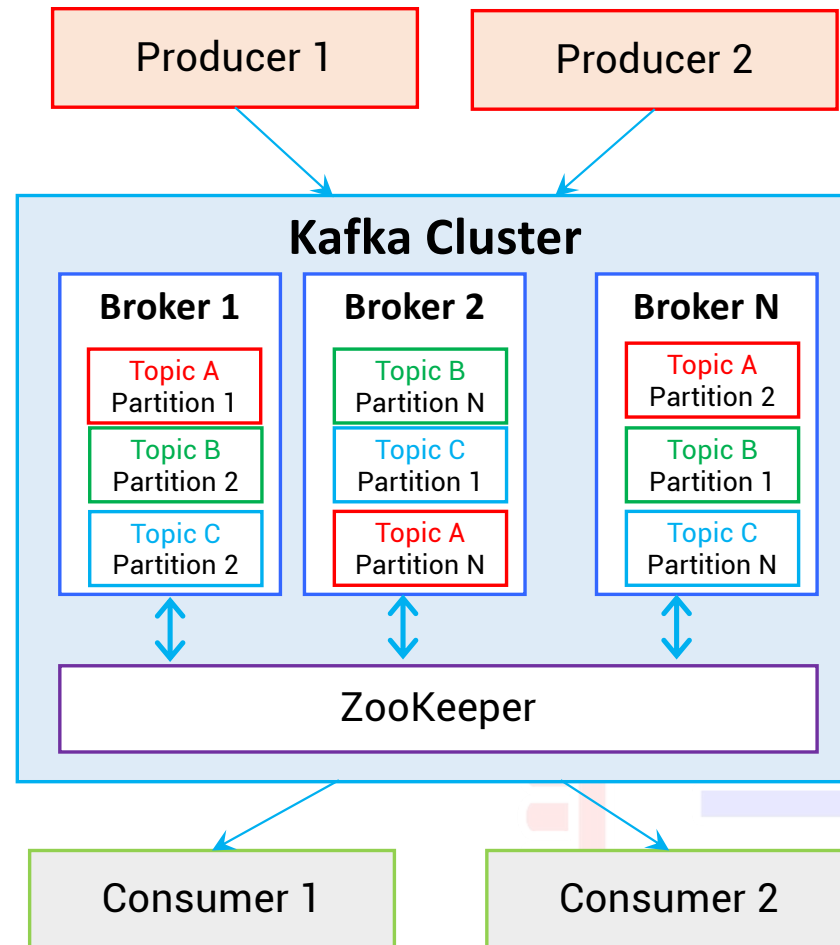
Apache Kafka : Architecture

■ Kafka Brokers

- Pour maintenir l'équilibre de la charge (un est élu comme Leader).
- besoin ZooKeeper pour conserver l'état du cluster.

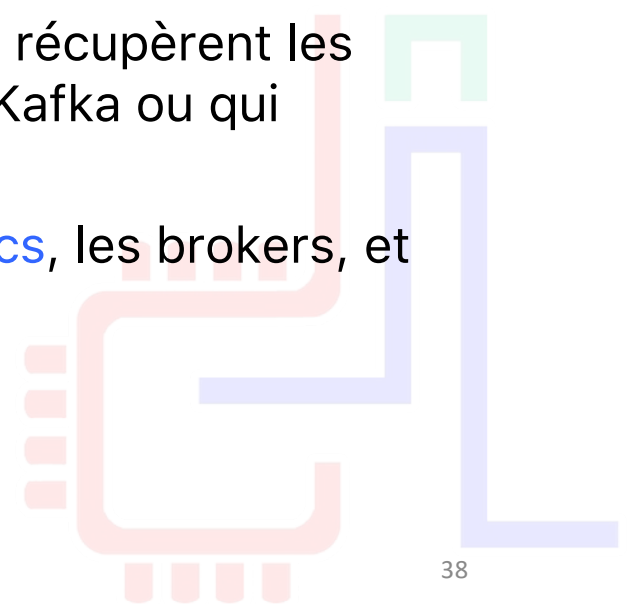
■ Zookeeper

- Pour la gestion de la coordination
- informer le producteur et le consommateur de la présence d'un nouveau broker



Apache Kafka : API

- **Producer API** permet aux applications d'envoyer des flux de données aux topics du cluster.
- **Consumer API** permet à une application de s'abonner à un ou plusieurs Topics et de traiter le flux d'enregistrements qui lui sont transmis.
- **Streams API** permet de transformer des flux de données d'entrée en topic de sortie.
- **Connect API** permet d'implémenter des connecteurs qui récupèrent les données d'un système source ou d'une application vers Kafka ou qui poussent de Kafka vers une application (SGBDR).
- **AdminClient API** permet de gérer et d'inspecter les topics, les brokers, et les autres objets Kafka .



Apache Kafka : Topics

- Un Topic est une **sorte de boîte à lettre** vers laquelle les messages (enregistrements) sont publiés
- Chaque Topic peut avoir 0 à plusieurs **abonnés consommateurs**
- Pour chaque topic, le cluster Kafka maintient un journal (Log) **partitionné**.
- Chaque partition est une séquence d'enregistrements ordonnée et immuable
- Un numéro d'identification séquentiel, appelé **offset**, est attribué aux enregistrements des partitions. Il identifie de **manière unique** chaque enregistrement de la partition.
- Le cluster Kafka **conserve durablement tous les enregistrements** publiés, qu'ils aient été consommés ou non, en utilisant une période de conservation **configurable**.
- Les performances de Kafka sont en réalité constantes en ce qui concerne la taille des données, de sorte que leur stockage pendant longtemps ne pose pas de problème.

Kafka : installation

- Installer Java

```
sudo apt-get install openjdk-8-jdk
```

- Installer zookeeper

```
sudo apt-get install zookeeperd
```

- Télécharger Kafka

```
wget https://downloads.apache.org/kafka/2.6.0/kafka_2.13-2.6.0.tgz
```

- Décompresser

```
sudo tar xvzf kafka_2.13-2.6.0.tgz
```

Kafka : utilisation

- Se positionner dans le répertoire `kafka/bin`

```
#Lancer Zookeeperavant Kafka
```

```
$/zookeeper-server-start.sh -daemon ../config/zookeeper.properties
```

```
#Lancer Kafka Server
```

```
$/kafka-server-start.sh -daemon ../config/server.properties
```

```
#Creation d'un topic nommé test
```

```
$ ./kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1  
--partitions 1 --topic test
```

```
Created topic test.
```

```
$ bin/kafka-topics.sh --list --zookeeper localhost 2181 test
```

```
#S'abonner au topic
```

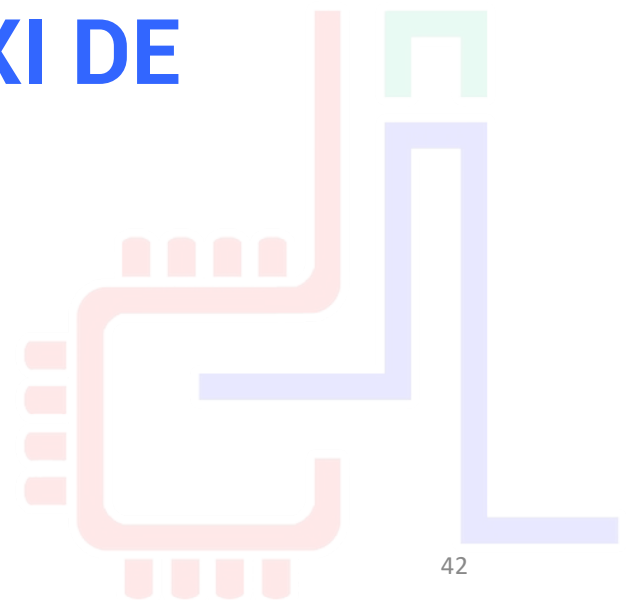
```
$ ./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test  
--from-beginning
```

```
#Produire des messages vers le topic
```

```
$ ./kafka-console-consumer.sh --bootstrap-server localhost:9092  
--topic test --from-beginning
```

TRAITER LES INFORMATIONS TAXI DE NEWYORK

Etude de cas



Lancement des serveurs

- Se positionner dans le répertoire `kafka/bin`

```
#Lancer Zookeeper
$./zookeeper-server-start.sh -daemon ../config/zookeeper.properties
#Lancer Kafka
$./kafka-server-start.sh -daemon ../config/server.properties
#Creation d un 1er topics
$./kafka-topics.sh \
--create --zookeeper localhost:2181 --replication-factor 1 \
--partitions 1 --topic taxirides
#Creation d un 2eme topics
$./kafka-topics.sh \
--create --zookeeper localhost:2181 --replication-factor 1 \
--partitions 1 --topic taxifares
```

- Topics créé en fonction de nos DataSets ici ils sont relatifs aux courses des taxis à New York

Nom	Date de modification	Taille	Type
 description.txt	aujourd'hui à 07:08	2 Ko	Format texte
 nycTaxiFares.gz	8 novembre 2020 à 10:55	19,5 Mo	archive...ressée
 nycTaxiRides.gz	8 novembre 2020 à 10:56	98,5 Mo	archive...ressée

Intégration des SSS avec Kafka

- Récupération des données de streamings depuis les topics Kafka

```
if __name__ == "__main__":

    spark = SparkSession.builder \
        .appName("Spark Structured Streaming from Kafka") \
        .getOrCreate()

    sdfRides = spark \
        .readStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", "localhost:9092") \
        .option("subscribe", "taxirides") \
        .option("startingOffsets", "latest") \
        .load() \
        .selectExpr("CAST(value AS STRING)")

    sdfFares = spark.readStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", "localhost:9092") \
        .option("subscribe", "taxifares") \
        .option("startingOffsets", "latest") \
        .load() \
        .selectExpr("CAST(value AS STRING)")
```

Souscription à un topic

L'option startingOffsets est réglé sur latest, le flux de données relancé lorsque l'application Spark attend des données.

Sélection de la colonne "value", contenant nos données de Taxi sous forme de la chaîne de caractères.

Extraction des données

- Préparation d'un schéma avant extraction des données

```
taxiFaresSchema = StructType([ \
    StructField("rideId", LongType()), StructField("taxiId", LongType()), \
    StructField("driverId", LongType()), StructField("startTime", TimestampType()), \
    StructField("paymentType", StringType()), StructField("tip", FloatType()), \
    StructField("tolls", FloatType()), StructField("totalFare", FloatType())
])

taxiRidesSchema = StructType([ \
    StructField("rideId", LongType()), StructField("isStart", StringType()), \
    StructField("endTime", TimestampType()), StructField("startTime", TimestampType()), \
    StructField("startLon", FloatType()), StructField("startLat", FloatType()), \
    StructField("endLon", FloatType()), StructField("endLat", FloatType()), \
    StructField("passengerCnt", ShortType())
])

def parse_data_from_kafka_message(sdf, schema):
    from pyspark.sql.functions import split
    assert sdf.isStreaming == True, "DataFrame doesn't receive streaming data"
    col = split(sdf['value'], ',') #split attributes to nested array in one Column
    #now expand col to multiple top-level columns
    for idx, field in enumerate(schema):
        sdf = sdf.withColumn(field.name, col.getItem(idx).cast(field.dataType))
    return sdf.select([field.name for field in schema])

sdfRides = parse_data_from_kafka_message(sdfRides, taxiRidesSchema)
sdfFares = parse_data_from_kafka_message(sdfFares, taxiFaresSchema)
```

1^{ère} Requête Streaming

```
query = sdfFares.groupBy("driverId").count()

query.writeStream \
  .outputMode("complete") \
  .format("console") \
  .option("truncate", False) \
  .start() \
  .awaitTermination()
```

- L'interface `DataFrame.writeStream` contrôle le comportement d'envoi de nos données streaming.
- 3 modes d'output déterminant quels résultats seront envoyés au collecteur de données externe :
 - `outputMode("complete")` Toutes les lignes sont utilisées lors du traitement de la requête agrégé (i.e count)
 - `outputMode("update")` pour considérer les nouvelles lignes et les lignes modifiés depuis le dernier trigger
 - `outputMode("append")` où seuls les nouvelles lignes seront prise en compte, précisément une fois. Les résultats précédents ne sont pas modifiables.
- Le format("console") : collecteur des données (Un autre topic Kafka, un fichier sink)

Lancement de l'application

```
$spark-submit \  
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.0.0 \  
ex_structured_streaming.py
```

```
-----  
Batch: 0  
-----
```

```
+-----+-----+  
|driverId|count|  
+-----+-----+  
+-----+-----+
```

- Ingestion des données Lancer les streams

```
(gzcatt /path/to/nycTaxiRides.gz \  
| gsplit -l 10000 --filter="./kafka-console-producer.sh \  
--broker-list localhost:9092 --topic taxirides; sleep 0.2"\  
> /dev/null ) &  
  
(gzcatt /path/to/nycTaxiFares.gz \  
| gsplit -l 10000 --filter="./kafka-console-producer.sh \  
--broker-list localhost:9092 --topic taxifares; sleep 0.2" \  
> /dev/null ) &
```

1^{er} Résultat

Batch: 5

driverId	count
2013005938	3
2013000047	3
2013005944	4
2013003666	1
2013000649	4
2013000656	3
2013008639	2
2013000894	4
2013009098	3
2013006543	2
2013008798	2
2013003820	4
2013001905	2
2013009095	2
2013003784	2
2013009796	1
2013006551	1
2013002065	5
2013007009	2
2013007438	3

only showing top 20 rows

Batch: 11

driverId	count
2013005938	6
2013000047	8
2013005944	7
2013003666	3
2013009973	2
2013010099	3
2013000649	5
2013000656	6
2013008639	3
2013000894	5
2013009098	8
2013006543	2
2013008798	6
2013003820	7
2013001905	2
2013009095	8
2013003784	9
2013009997	6
2013009796	5
2013006551	4

only showing top 20 rows

Batch: 29

driverId	count
2013010867	5
2013005938	12
2013000047	18
2013005944	15
2013016275	2
2013003666	14
2013015881	3
2013014386	3
2013010882	16
2013009973	10
2013010099	12
2013000649	5
2013000656	23
2013008639	3
2013012838	8
2013016398	3
2013000894	10
2013009098	15
2013006543	2
2013008798	12

only showing top 20 rows

2^{ème} Exemple : jointure de Stream

- Limiter le traitement sur la zone de Manhattan

```
LON_EAST, LON_WEST, LAT_NORTH, LAT_SOUTH = -73.7, -74.05, 41.0, 40.5
sdfRides = sdfRides.filter( \
    sdfRides["startLon"].between(LON_WEST, LON_EAST) & \
    sdfRides["startLat"].between(LAT_SOUTH, LAT_NORTH) & \
    sdfRides["endLon"].between(LON_WEST, LON_EAST) & \
    sdfRides["endLat"].between(LAT_SOUTH, LAT_NORTH))
sdfRides = sdfRides.filter(sdfRides["isStart"] == "END") #Keep only finished
```

- Watermarks

```
# Apply watermarks on event-time columns
sdfFaresWithWatermark = sdfFares \
    .selectExpr("rideId AS rideId_fares", "driverId", "taxiId", "startTime", "totalFare", "tip") \
    .withWatermark("startTime", "30 minutes") # maximal delay

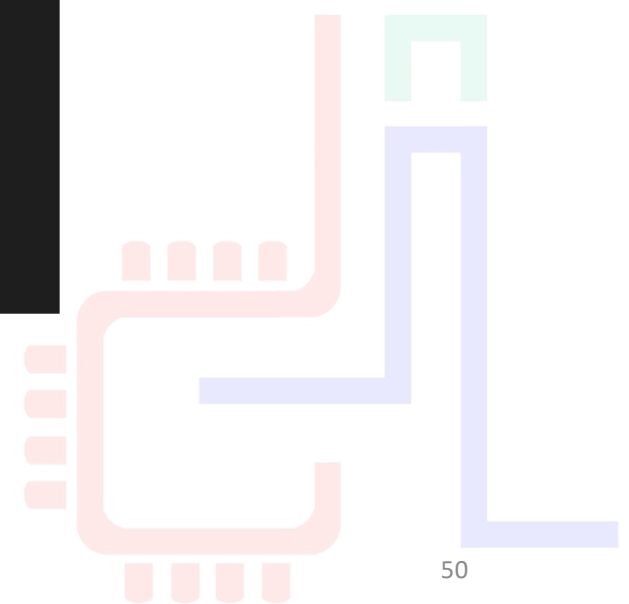
sdfRidesWithWatermark = sdfRides \
    .selectExpr("rideId", "endTime", \
        "startLon", "startLat", "endLon", "endLat") \
    .withWatermark("endTime", "30 minutes") # maximal delay
```

2^{ème} Exemple : jointure de Stream

- La jointure

```
# Join with event-time constraints
sdf = sdfFaresWithWatermark \
    .join(sdfRidesWithWatermark, \
        expr('''rideId_fares == rideId AND
              endTime > startTime AND
              endTime <= startTime + interval 2 hours'''))

sdf.writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", False) \
    .start() \
    .awaitTermination()
```



Résultat

Batch: 1

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
--------------	----------	--------	-----------	-----------	-----	--------	---------	----------	----------	--------	--------

Batch: 7

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
51662	2013009823	2013009823	2013-01-01 01:41:58	4.55	1.05	51662	2013-01-01 03:05:00	-73.99825	40.754307	-73.99632	40.758247

Batch: 11

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
135646	2013003187	2013003190	2013-01-01 05:09:48	10.5	0.0	135646	2013-01-01 05:48:00	-73.988495	40.727325	-73.90709	40.768463

Batch: 14

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
189674	2013012830	2013007608	2013-01-01 11:26:45	9.5	1.0	189674	2013-01-01 11:37:58	-73.98991	40.73101	-73.97017	40.799515
177901	2013013963	2013006920	2013-01-01 10:32:00	41.5	6.0	177901	2013-01-01 11:56:20	-74.00143	40.74645	-73.98053	40.7302

Batch: 51

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
677287	2013006371	2013006375	2013-01-02 18:01:00	8.5	0.0	677287	2013-01-02 19:19:02	-73.99108	40.7394	-73.97911	40.721878
677964	2013024259	2013003154	2013-01-02 18:02:40	6.5	0.0	677964	2013-01-02 19:10:33	-73.97724	40.75087	-73.980545	40.729855
705488	2013024142	2013003959	2013-01-02 19:02:33	12.0	2.0	705488	2013-01-02 19:26:00	-73.98861	40.731174	-73.994774	40.73897
667977	2013002367	2013002370	2013-01-02 17:38:03	10.2	1.7	667977	2013-01-02 19:19:02	-73.98852	40.763275	-74.001045	40.74238
666578	2013008817	2013008821	2013-01-02 17:35:00	19.25	3.75	666578	2013-01-02 19:32:59	-73.98964	40.735302	-73.991196	40.758858

Résultat

Batch: 0

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
--------------	----------	--------	-----------	-----------	-----	--------	---------	----------	----------	--------	--------

Batch: 2

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
677287	2013006371	2013006375	2013-01-02 18:01:00	8.5	0.0	677287	2013-01-02 19:19:02	-73.99108	40.7394	-73.97911	40.721878
677964	2013024259	2013003154	2013-01-02 18:02:40	6.5	0.0	677964	2013-01-02 19:10:33	-73.97724	40.75087	-73.980545	40.729855
705488	2013024142	2013003959	2013-01-02 19:02:33	12.0	2.0	705488	2013-01-02 19:26:00	-73.98861	40.731174	-73.994774	40.73897

Batch: 4

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
757663	2013006501	2013006505	2013-01-02 21:13:03	8.5	1.0	757663	2013-01-02 18:00:50	-73.977974	40.749065	-73.99216	40.74426
753349	2013024177	2013007153	2013-01-02 21:01:00	11.5	0.0	753349	2013-01-02 19:36:37	-73.97438	40.762123	-73.97777	40.75817
750948	2013008964	2013007493	2013-01-02 20:54:00	9.25	1.75	750948	2013-01-02 18:06:14	-73.98905	40.77437	-73.98094	40.76534
678948	2013006867	2013006871	2013-01-02 18:05:00	6.5	0.0	678948	2013-01-02 20:14:05	-73.95175	40.781784	-73.951454	40.804497

Batch: 9

rideId_fares	driverId	taxiId	startTime	totalFare	tip	rideId	endTime	startLon	startLat	endLon	endLat
766572	2013001890	2013001893	2013-01-02 21:39:19	7.0	0.0	766572	2013-01-03 02:03:19	-73.99058	40.72466	-73.99681	40.744663
678974	2013023647	2013012745	2013-01-02 18:05:00	10.5	0.0	678974	2013-01-03 03:33:57	-74.00596	40.743767	-73.94719	40.764442
753272	2013001699	2013003515	2013-01-02 21:00:52	12.0	2.0	753272	2013-01-03 03:12:52	-74.006546	40.72549	-74.00365	40.73413
680195	2013018344	2013001023	2013-01-02 18:07:59	16.0	0.0	680195	2013-01-03 04:33:40	-73.959114	40.80117	-73.872765	40.77434
755811	2013025052	2013008676	2013-01-02 21:08:00	9.5	0.0	755811	2013-01-03 01:51:13	-73.98338	40.73161	-73.97732	40.738728
848626	2013019938	2013007641	2013-01-03 07:11:33	5.0	0.0	848626	2013-01-02 20:13:00	-73.7902	40.644104	-73.97552	40.765526
684713	2013009728	2013009729	2013-01-02 18:18:00	13.5	0.0	684713	2013-01-03 02:00:46	-73.98502	40.727783	-73.99322	40.722706
842280	2013020172	2013011001	2013-01-03 06:45:34	10.62	2.12	842280	2013-01-03 02:45:00	-74.00234	40.734646	-74.00424	40.707726
841610	2013015909	2013004909	2013-01-03 06:43:00	14.25	2.25	841610	2013-01-02 20:57:00	-73.87729	40.77241	-73.97707	40.67921
767379	2013000317	2013000317	2013-01-02 21:41:58	6.1	1.1	767379	2013-01-03 03:52:01	-73.98792	40.724148	-73.942894	40.68939